ARRETADO 2.0

MANUAL COMPLETO E ATUALIZADO

Projetos 1 e 2 • Linguagem esotérica nordestina • Tradução de .arr para Python

Edição consolidada | Outubro de 2026 | Guia de instalação, arquitetura, comandos, códigos integrais, testes e entrega

Objetivo. Este material descreve fielmente a versão corrigida do pacote ARRETADO usada nos testes. Mostra o que já funciona, o que ainda precisa ser aprimorado e como reproduzir os resultados no Windows com Python 3.14.

Sumário de aprendizagem

• Parte A: o que são os projetos e como o sistema é organizado.

• Parte B: instalação, VS Code, terminal e tradução.

• Parte C: comandos e regras oficiais da ARRETADO 2.0.

• Parte D: os três programas do Projeto 1, com códigos integrais.

• Parte E: os cinco programas do Projeto 2, com códigos integrais.

• Parte F: testes, erros conhecidos, boas práticas, apresentação e checklist.

1. O que o professor solicitou

Projeto 1 (entrega em 18/11/2026): inventar uma linguagem esotérica, sua extensão e documentação; implementar tradutor para linguagem conhecida; entregar Hello World, 99 garrafas de cerveja e programa livre; explicar como compilar/traduzir e executar. Avaliação: documentação 3, tradutor 3, programas 1, execução 3 pontos.

Projeto 2 (entrega em 02/12/2026): desenvolver, na mesma linguagem, fatorial até 15, N-ésimo Fibonacci de 1 a 10, Torre de Hanói de 6 discos, ordenação de N inteiros com N menor que 100 e CRUD persistido em JSON. Entregar os fontes e as instruções; nenhum recurso Python deve ser usado diretamente nos programas .arr fora da linguagem definida.

2. Qual é a ARRETADO?

ARRETADO é uma linguagem esotérica didática com comandos inspirados em expressões nordestinas. Seus programas usam a extensão .arr. O arquivo tradutor.py lê o fonte .arr, verifica uma gramática de comandos definida e gera um arquivo .py. O interpretador Python executa esse arquivo gerado.

Fluxo: editar arquivo .arr → salvar → traduzir com tradutor.py → gerar arquivo .py → executar com Python → conferir a saída.

É um tradutor fonte-para-fonte (transpilador), não um compilador de código de máquina. O Python funciona como linguagem de destino. A originalidade deve ser defendida pelas regras da ARRETADO, seus comandos próprios, validação sintática e funções de manipulação de listas e JSON.

3. Instalar as ferramentas

• Windows: instalar Python 3.10 ou superior; no computador de testes, Python 3.14 funcionou.

• Instalar VS Code. Extensão Python do VS Code é útil, mas não é obrigatória.

• Extrair o ZIP corrigido completo e abrir a subpasta ARRETADO em Arquivo > Abrir Pasta.

• Abrir Terminal > Novo Terminal e usar python --version e Get-Location.

• Nenhum pacote de terceiros é necessário para o tradutor: usa bibliotecas padrão do Python.

```
python --version
Get-Location
Get-ChildItem
```

Se o terminal terminar em ...\ARRETADO_Projetos_1_e_2_Corrigido_Completo, use cd ARRETADO. Os scripts estão dentro da pasta ARRETADO, não na pasta superior.

4. Estrutura de pastas

```
ARRETADO/
  tradutor.py                # tradutor .arr -> .py
  testar.py                  # verifica os 3 exemplos do Projeto 1
  testar_tudo.py             # testes dos Projetos 1 e 2
  README.md                  # instrucoes resumidas
  exemplos/
    ola_mundo.arr
    99_garrafas.arr
    programa_livre.arr
  projeto2/
    01_fatorial.arr
    02_fibonacci.arr
    03_hanoi.arr
    04_ordenacao.arr
    05_crud_json.arr
  gerados/                   # programas Python gerados
  registros.json             # criado quando CRUD salva (conforme pasta atual)
```

5. Primeira execução: testes automáticos

```
cd ARRETADO                # somente se ainda estiver na pasta superior
python testar.py
python testar_tudo.py
```

Os testes que você executou no Windows retornaram:

```
[OK] ola_mundo
[OK] 99_garrafas
[OK] programa_livre
Todos os tres testes passaram.

TODOS OS TESTES PASSARAM: 3 programas do Projeto 1 + 5 do Projeto 2, limites e
validacao.
```

Testes automáticos confirmam cenários programados; não substituem conferir manualmente entradas inválidas, arquivos JSON corrompidos ou fazer uma apresentação da ferramenta.

6. Regras de programação

• Todo programa começa por OXENTE e termina por CABOU, únicos, nessa ordem.

• Linhas em branco são permitidas; comentários começam por #.

• Identificadores usam letras ASCII, números e _, sem começar por número ou _.

• Blocos são fechados por FIM_ENQUANTO, FIM_SE ou FIM_FUNCAO. A indentação organiza a leitura; os delimitadores definem o bloco.

• Os comandos devem estar escritos exatamente como a linguagem define, com a grafia e os argumentos corretos.

• Expressões aceitam números inteiros e decimais, strings entre aspas, booleanos, listas e acesso por índice.

• Operadores: +, -, *, /, //, %, comparações == != < <= > >= e lógicos and, or, not.

• Funções precisam ser definidas no nível principal antes das chamadas realizadas; podem ser recursivas.

7. Dicionário completo de comandos

Comando	Como usar / finalidade
OXENTE / CABOU	Inicia / encerra fonte .arr
ARRE_EGUA expr	Mostra resultado de uma expressão
GUARDA x = expr	Cria/atualiza uma variável
LEIA_NUM x / LEIA_TEXTO x	Lê inteiro / texto do teclado
SE_AVEXE condição	Início da decisão
NUM_DEU / FIM_SE	Caso contrário / fechamento da decisão
ENQUANTO_TIVER condição	Início de laço
FIM_ENQUANTO	Fecha laço
MAIS_UM x / MENOS_UM x	Soma / subtrai 1
PONHA lista expr	Adiciona elemento ao fim
TROQUE lista indice valor	Substitui um elemento
TIRE lista indice	Remove elemento por índice
FUNCAO nome(params)	Declara função própria
DEVOLVE expr / CHAMA nome(args)	Retorna valor / chama função
FIM_FUNCAO	Encerra função
JSON_CARREGA var "arq.json"	Lê JSON ou cria lista vazia se ausente

Comando	Como usar / finalidade
JSON_SALVA "arq.json" var	Grava objeto em JSON
TAMANHO(), TEXTO(), NUMERO()	Obtém tamanho / converte a texto / inteiro

8. Mini aula: variáveis, leitura e decisões

```
OXENTE
    ARRE_EGUA "Digite um numero inteiro:"
    LEIA_NUM numero
    SE_AVEXE numero >= 18
        ARRE_EGUA "Maior ou igual a 18"
    NUM_DEU
        ARRE_EGUA "Menor que 18"
    FIM_SE
CABOU
```

Salve esse exemplo como exemplos/idade.arr. O comando LEIA_NUM converte a entrada em inteiro; se você pressionar Enter em branco ou digitar letras, poderá ocorrer ValueError. Essa é uma limitação atual, não um recurso já corrigido.

9. Como traduzir e executar qualquer .arr

```
python tradutor.py exemplos/ola_mundo.arr -o gerados/ola_mundo.py
python gerados/ola_mundo.py
```

A primeira instrução cria o código Python. A segunda efetivamente executa o programa. Sempre traduza novamente depois de editar e salvar o .arr. Rodar tradutor.py sem informar arquivo exibe o aviso de argumento obrigatório; é uso incorreto do comando, não falha do tradutor.

10. Criar um programa do zero

• No explorador do VS Code, clique com o botão direito em exemplos e escolha Novo Arquivo.

• Digite teste_pessoal.arr (a extensão é obrigatória).

• Cole o exemplo abaixo, salve com Ctrl+S e traduza pelo terminal.

• Abra gerados/teste_pessoal.py para visualizar como ficou o Python produzido.

• Modifique os textos e valores e compare as saídas.

```
OXENTE
    ARRE_EGUA "Oxe, eu programei!"
    GUARDA contador = 1
    ENQUANTO_TIVER contador <= 5
        ARRE_EGUA contador
        MAIS_UM contador
    FIM_ENQUANTO
CABOU

# No terminal:
# python tradutor.py exemplos/teste_pessoal.arr -o gerados/teste_pessoal.py
# python gerados/teste_pessoal.py
```

11. Projeto 1: instruções de reprodução

ola_mundo.arr — Exibe Hello World!

```
python tradutor.py exemplos/ola_mundo.arr -o gerados/ola_mundo.py
python gerados/ola_mundo.py
```

99_garrafas.arr — Inicia em 99 e reduz até zero, com mensagem de singular/plural.

```
python tradutor.py exemplos/99_garrafas.arr -o gerados/99_garrafas.py
python gerados/99_garrafas.py
```

programa_livre.arr — Soma 2 vezes cada rodada, de 1 a 5: pontuação final 30.

```
python tradutor.py exemplos/programa_livre.arr -o gerados/programa_livre.py
python gerados/programa_livre.py
```

12. Projeto 2: fatorial

Problema: dado N natural de 0 a 15, calcular N! = 1×2×...×N. Por definição, 0!=1. Solução: multiplicador inicial 1 e laço de 2 até N. Seu teste manual: N=6 resultou 720, correto.

```
python tradutor.py projeto2/01_fatorial.arr -o gerados/01_fatorial.py
python gerados/01_fatorial.py
```

13. Projeto 2: Fibonacci

Convenção adotada no fonte: F(1)=0 e F(2)=1. Cada próximo termo é a soma dos dois anteriores. Entradas válidas: de 1 a 10. Seu teste: N=4 retornou 2, consistente com essa convenção.

```
python tradutor.py projeto2/02_fibonacci.arr -o gerados/02_fibonacci.py
python gerados/02_fibonacci.py
```

14. Projeto 2: Torre de Hanói

Seis discos, três pinos A/B/C, deslocando um disco por vez e sem colocar maior sobre menor. Uma função recursiva move N-1 discos ao auxiliar, move o maior e desloca N-1 ao destino. Mínimo de 2^6-1 = 63 movimentos. Você exibiu os 63 movimentos de A a C.

```
python tradutor.py projeto2/03_hanoi.arr -o gerados/03_hanoi.py
python gerados/03_hanoi.py
```

15. Projeto 2: ordenação

O programa lê N de 0 a 99 e, em seguida, N inteiros. Usa Bubble Sort, comparando números adjacentes e trocando suas posições quando necessário. Seu teste de 10 números resultou em [8, 9, 12, 13, 14, 15, 16, 17, 18, 20].

```
python tradutor.py projeto2/04_ordenacao.arr -o gerados/04_ordenacao.py
python gerados/04_ordenacao.py
```

16. Projeto 2: CRUD JSON

O arquivo registros.json armazena uma lista JSON de nomes. Criar adiciona string; listar percorre registros; atualizar substitui por índice; excluir remove pelo índice. O sistema salva após as operações que mudam dados.

```
python tradutor.py projeto2/05_crud_json.arr -o gerados/05_crud_json.py
python gerados/05_crud_json.py
```

Roteiro de teste: escolha 1, informe um nome; escolha 2 e veja a lista; escolha 3, informe índice e nome atualizado; escolha 4 e exclua; escolha 2 para verificar; escolha 0 para sair. Reinicie e confira a persistência.

Teste que você realizou: cadastrou agosto [sic: no seu teste, o nome foi "augusto"], listou ahghs e augusto, atualizou o índice 0 para teste, removeu o índice 1, e a listagem final mostrou 0: teste. Isso demonstra as operações e persistência entre execuções.

Atenção: LEIA_NUM recebe números inteiros válidos. Uma linha vazia no menu provocou ValueError no Windows. O CRUD não está à prova de erros; não declare que a entrada inválida foi corrigida. Um JSON malformado também pode causar falha de execução.

17. Erros e soluções no Windows

Mensagem	Causa e ação
can’t open file testar.py	Você está no diretório superior. Digite cd ARRETADO e verifique Get-ChildItem.
can’t open file testar.pyv	Nome digitado com v adicional. Use python testar.py.
UnicodeDecodeError em subprocess	Scripts antigos não tratavam saída UTF-8 no Windows; use a versão corrigida do ZIP.
ValueError ... int() ... vazio	LEIA_NUM recebeu Enter vazio. Digite 0–4; validação ainda é uma melhoria pendente.
tradutor.py: arquivo required	Informe o .arr: python tradutor.py exemplos/ola_mundo.arr -o gerados/ola_mundo.py.
Comando desconhecido na linha X	Verifique a lista oficial, os argumentos, fechamentos e letras maiúsculas.
Índice inválido no CRUD	O programa verifica posições fora da lista para as operações 3 e 4.

18. Testes manuais que já foram confirmados

Cenário	Resultado observado
testar.py	3 exemplos do Projeto 1 aprovados
testar_tudo.py	3 programas do Projeto 1 + 5 do Projeto 2 aprovados
Hello World	Hello World!
Fatorial 6	720

Cenário	Resultado observado
Fibonacci 4	2
Hanói com 6 discos	63 movimentos válidos exibidos
Ordenação de 10 inteiros	[8, 9, 12, 13, 14, 15, 16, 17, 18, 20]
CRUD	Criou, listou, atualizou, excluiu e persistiu registros

19. O que NÃO está implementado ainda

• Um executador integrado arretado.py que traduza e rode em um único comando. Atualmente há duas etapas.

• Tratamento completo de Enter vazio e entradas não numéricas em LEIA_NUM.

• Classes, dicionários literais, imports externos e try/except em código .arr.

• Recuperação automática de conteúdo JSON inválido ou corrompido.

• Ambiente visual próprio, depurador próprio ou executável independente de Python.

Essas limitações devem ser comunicadas com transparência se forem perguntadas na apresentação. O professor avaliará o conjunto; testes positivos não garantem nota máxima.

20. Como explicar o tradutor ao professor

• Arquivo de entrada: o tradutor recebe um caminho .arr e lê em UTF-8.

• Pré-verificação: reconhece cabeçalho OXENTE, encerramento CABOU, funções declaradas e comandos permitidos.

• Expressões: analisa a árvore sintática usando ast e permite somente tipos e operações selecionadas.

• Estruturas: mantém uma pilha para conferir abertura/fechamento de condicionais, laços e funções.

• Emissão: cria código Python equivalente, com funções auxiliares de JSON implementadas no runtime gerado.

• Saída: grava o .py especificado com -o. O interpretador Python executa esse arquivo.

Por que o CRUD não viola a regra de evitar Python no código-fonte? Porque o fonte .arr só usa JSON_CARREGA e JSON_SALVA, comandos definidos na ARRETADO; o tradutor implementa seus efeitos em Python, assim como implementa impressão, laços e listas.

21. Sugestão de divisão do trabalho em grupo

• Integrante A: ideia da linguagem, especificação de sintaxe e exemplos curtos.

• Integrante B: análise sintática e tradução dos comandos.

• Integrante C: programas de algoritmos e testes de limites.

• Integrante D: arquivos JSON, documentação, evidências e apresentação.

• Todos: revisar se cada comando consta da documentação e saber executar qualquer programa.

22. Checklist antes de enviar ao SIGAA

• Confirmar os arquivos do Projeto 1: manual, tradutor, três .arr e instruções.

• Confirmar os cinco .arr do Projeto 2 e passos completos de execução.

• Rodar python testar.py e python testar_tudo.py na máquina de apresentação.

• Demonstrar fatorial, Fibonacci, 63 movimentos da Hanói, ordenação e CRUD.

• Conferir se cada comando de cada fonte .arr está documentado.

• Evitar entregar registros.json com dados pessoais reais ou registros de teste indevidos.

• Verificar se todos os membros do grupo compreendem o tradutor e os exemplos.

• Respeitar prazos: Projeto 1 em 18/11/2026 e Projeto 2 em 02/12/2026.

23. Anexo A: todos os programas .arr, na íntegra

Os códigos a seguir foram copiados dos arquivos do pacote corrigido, sem reescrita simplificadora. Essa seção permite estudar cada linha e reconstituir os fontes.

exemplos/99_garrafas.arr

```
# A canção das 99 garrafas - versão simplificada
OXENTE
GUARDA garrafas = 99
ENQUANTO_TIVER garrafas > 0
    ARRE_EGUA garrafas
    SE_AVEXE garrafas == 1
        ARRE_EGUA "garrafa de cerveja na parede."
    NUM_DEU
        ARRE_EGUA "garrafas de cerveja na parede."
    FIM_SE
    ARRE_EGUA "Tire uma, passe adiante!"
    MENOS_UM garrafas
FIM_ENQUANTO
ARRE_EGUA "Nenhuma garrafa de cerveja na parede!"
CABOU
```

exemplos/ola_mundo.arr

```
# Primeiro programa ARRETADO
OXENTE
ARRE_EGUA "Hello World!"
CABOU
```

exemplos/programa_livre.arr

```
# Mini-tabuleiro de pontuação nordestina
OXENTE
GUARDA pontos = 0
GUARDA rodada = 1
ENQUANTO_TIVER rodada <= 5
    ARRE_EGUA "Rodada:"
    ARRE_EGUA rodada
```

```
    GUARDA pontos = pontos + rodada * 2
    MAIS_UM rodada
FIM_ENQUANTO
SE_AVEXE pontos >= 20
    ARRE_EGUA "Arretado! Meta alcançada."
NUM_DEU
    ARRE_EGUA "Bora tentar de novo!"
FIM_SE
ARRE_EGUA "Pontuação final:"
ARRE_EGUA pontos
CABOU
```

projeto2/01_fatorial.arr

```
# Fatorial de N entre 0 e 15
OXENTE
ARRE_EGUA "Digite N (0 a 15):"
LEIA_NUM n
SE_AVEXE n < 0 or n > 15
    ARRE_EGUA "N fora do intervalo permitido"
NUM_DEU
    GUARDA produto = 1
    GUARDA i = 2
    ENQUANTO_TIVER i <= n
        GUARDA produto = produto * i
        MAIS_UM i
    FIM_ENQUANTO
    ARRE_EGUA produto
FIM_SE
CABOU
```

projeto2/02_fibonacci.arr

```
# Sequência começa em F(1)=0 e F(2)=1
OXENTE
ARRE_EGUA "Digite N (1 a 10):"
LEIA_NUM n
SE_AVEXE n < 1 or n > 10
    ARRE_EGUA "N fora do intervalo permitido"
NUM_DEU
    GUARDA a = 0
    GUARDA b = 1
    GUARDA i = 1
    ENQUANTO_TIVER i < n
        GUARDA proximo = a + b
        GUARDA a = b
        GUARDA b = proximo
        MAIS_UM i
    FIM_ENQUANTO
    ARRE_EGUA a
FIM_SE
CABOU
```

projeto2/03_hanoi.arr

```
# Estratégia recursiva ótima: 2^6 - 1 = 63 movimentos
OXENTE
FUNCAO mover(n, origem, destino, auxiliar)
    SE_AVEXE n == 1
        ARRE_EGUA "Disco 1: " + origem + " -> " + destino
    NUM_DEU
        CHAMA mover(n - 1, origem, auxiliar, destino)
        ARRE_EGUA "Disco " + TEXTO(n) + ": " + origem + " -> " + destino
        CHAMA mover(n - 1, auxiliar, destino, origem)
    FIM_SE
FIM_FUNCAO
CHAMA mover(6, "A", "C", "B")
```

```
CABOU
```

projeto2/04_ordenacao.arr

```
# Bubble sort para N < 100 inteiros
OXENTE
ARRE_EGUA "Digite N (0 a 99):"
LEIA_NUM n
SE_AVEXE n < 0 or n >= 100
    ARRE_EGUA "N fora do intervalo permitido"
NUM_DEU
    GUARDA numeros = []
    GUARDA i = 0
    ENQUANTO_TIVER i < n
        LEIA_NUM valor
        PONHA numeros valor
        MAIS_UM i
    FIM_ENQUANTO
    GUARDA i = 0
    ENQUANTO_TIVER i < n
        GUARDA j = 0
        ENQUANTO_TIVER j < n - 1 - i
            SE_AVEXE numeros[j] > numeros[j + 1]
                GUARDA temp = numeros[j]
                TROQUE numeros j numeros[j + 1]
                TROQUE numeros (j+1) temp
            FIM_SE
            MAIS_UM j
        FIM_ENQUANTO
        MAIS_UM i
    FIM_ENQUANTO
    ARRE_EGUA numeros
FIM_SE
CABOU
```

projeto2/05_crud_json.arr

```
# CRUD de nomes persistente em registros.json; sem Python no código .arr
OXENTE
JSON_CARREGA registros "registros.json"
GUARDA opcao = -1
ENQUANTO_TIVER opcao != 0
    ARRE_EGUA "0=Sair  1=Criar  2=Listar  3=Atualizar  4=Excluir"
    LEIA_NUM opcao
    SE_AVEXE opcao == 1
        ARRE_EGUA "Nome para adicionar:"
        LEIA_TEXTO nome
        PONHA registros nome
        JSON_SALVA "registros.json" registros
        ARRE_EGUA "Adicionado"
    FIM_SE
    SE_AVEXE opcao == 2
        GUARDA i = 0
        ENQUANTO_TIVER i < TAMANHO(registros)
            ARRE_EGUA TEXTO(i) + ": " + registros[i]
            MAIS_UM i
        FIM_ENQUANTO
    FIM_SE
    SE_AVEXE opcao == 3
        ARRE_EGUA "Índice a atualizar:"
        LEIA_NUM indice
        SE_AVEXE indice >= 0 and indice < TAMANHO(registros)
            ARRE_EGUA "Novo nome:"
            LEIA_TEXTO nome
            TROQUE registros indice nome
            JSON_SALVA "registros.json" registros
            ARRE_EGUA "Atualizado"
```

```
        NUM_DEU
            ARRE_EGUA "Índice inválido"
        FIM_SE
    FIM_SE
    SE_AVEXE opcao == 4
        ARRE_EGUA "Índice a excluir:"
        LEIA_NUM indice
        SE_AVEXE indice >= 0 and indice < TAMANHO(registros)
            TIRE registros indice
            JSON_SALVA "registros.json" registros
            ARRE_EGUA "Excluído"
        NUM_DEU
            ARRE_EGUA "Índice inválido"
        FIM_SE
    FIM_SE
FIM_ENQUANTO
CABOU
```

24. Anexo B: roteiro de demonstração em 5 minutos

00:00–00:45: apresentar nome, inspiração regional, extensão .arr e arquitetura. 00:45–01:30: mostrar Hello World e traduzir. 01:30–02:30: executar fatorial e Fibonacci. 02:30–03:15: mostrar Hanói e ordenação. 03:15–04:30: CRUD completo e abrir registros.json. 04:30–05:00: mostrar testes e limitações conhecidas.

25. Referências de origem e escopo desta edição

Base: enunciados Projeto 1.pdf e Projeto 2.pdf apresentados na conversa, pacote ARRETADO_Projetos_1_e_2_Corrigido_Completo.zip (tradutor, README e oito programas .arr), além dos resultados de execução que você compartilhou no PowerShell. Este manual documenta o estado funcional conhecido em outubro de 2026; melhorias propostas estão identificadas como não implementadas.